

Computer Programming Summer Interest Group

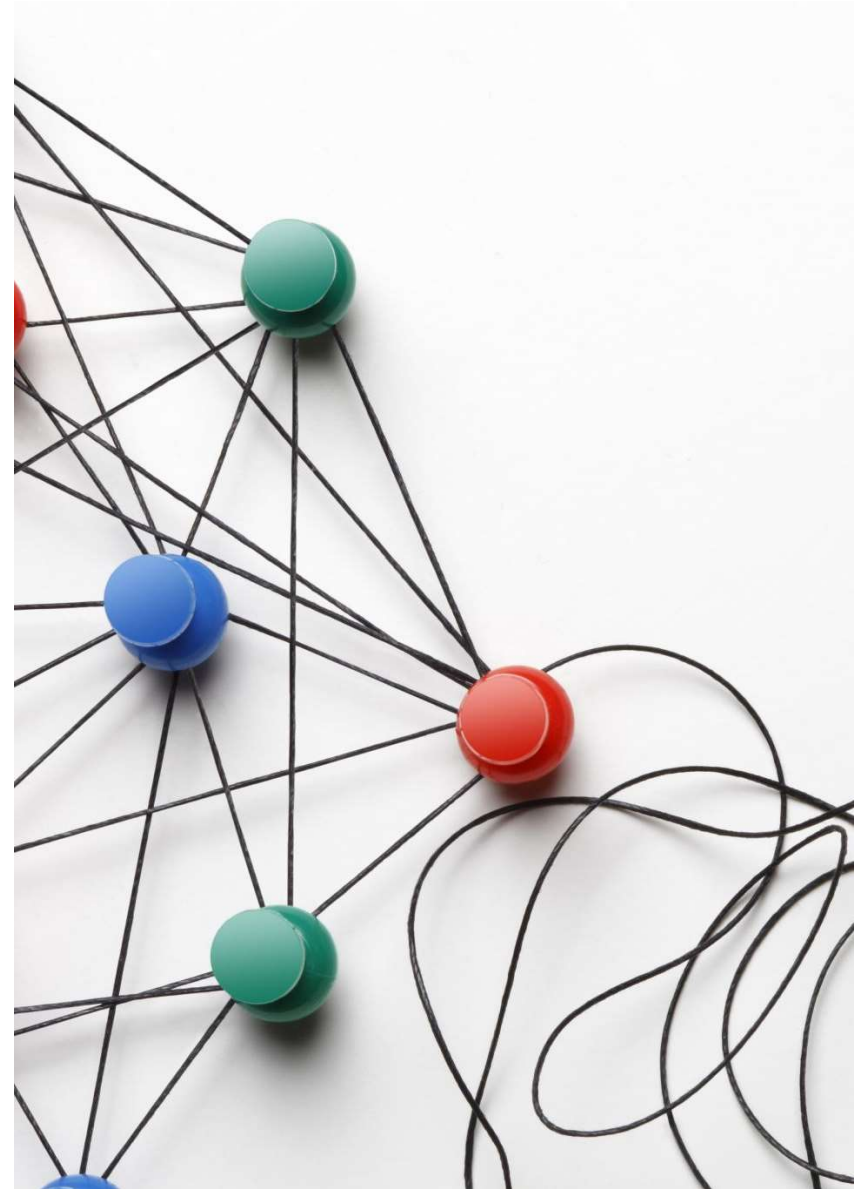
Session 11 / Errors

Matthew 5:21-26

“You have heard that it was said to those of old, ‘You shall not murder; and whoever murders will be liable to judgment.’ But I say to you that everyone who is angry with his brother will be liable to judgment; whoever insults his brother will be liable to the council; and whoever says, ‘You fool!’ will be liable to the hell of fire. So if you are offering your gift at the altar and there remember that your brother has something against you, leave your gift there before the altar and go. First be reconciled to your brother, and then come and offer your gift. Come to terms quickly with your accuser while you are going with him to court, lest your accuser hand you over to the judge, and the judge to the guard, and you be put in prison. Truly, I say to you, you will never get out until you have paid the last penny.

Objectives

- By the end of this session, you will be able to:
 - Understand what errors and exceptions are in Python.
 - Differentiate between syntax errors and runtime exceptions.
 - Use try, except, else, and finally blocks.
 - Raise exceptions using raise.
 - Use runtime assertions
 - Apply error handling to make programs more robust.



Sources of errors

Invalid input

- User input can be improperly formatted or have bad content (e.g. entering “hi” when a number is expected)
- A referenced file may not have the correct format
- A referenced file may contain invalid values

Unexpected situations

- A requested file or other resource may not exist
- A requested file or other resource may have permissions preventing access

Incorrect programming logic

- Bugs in a program will likely produce incorrect / invalid results

Python Exceptions

- Unhandled exceptions will cause the program to terminate prematurely
 - Many internal python functions / methods will raise exceptions
 - A programmer may use exceptions to handle error states in new code
- Exceptions allow a program to indicate when an error state occurs
 - Generally used when the logic cannot continue due to the error state
- Exceptions allow a program to recover from an error state
 - Allows the program to trap the error without causing the program to halt



What could go wrong?

- User may not enter an integer value
 - Will raise a ValueError
- User may enter 0
 - Will raise a divide by 0 error
- These will both raise exceptions
 - The program will terminate with an error message to the console

```
x = input("Enter a number: ")
x = int(x)
print(10 / x)
```

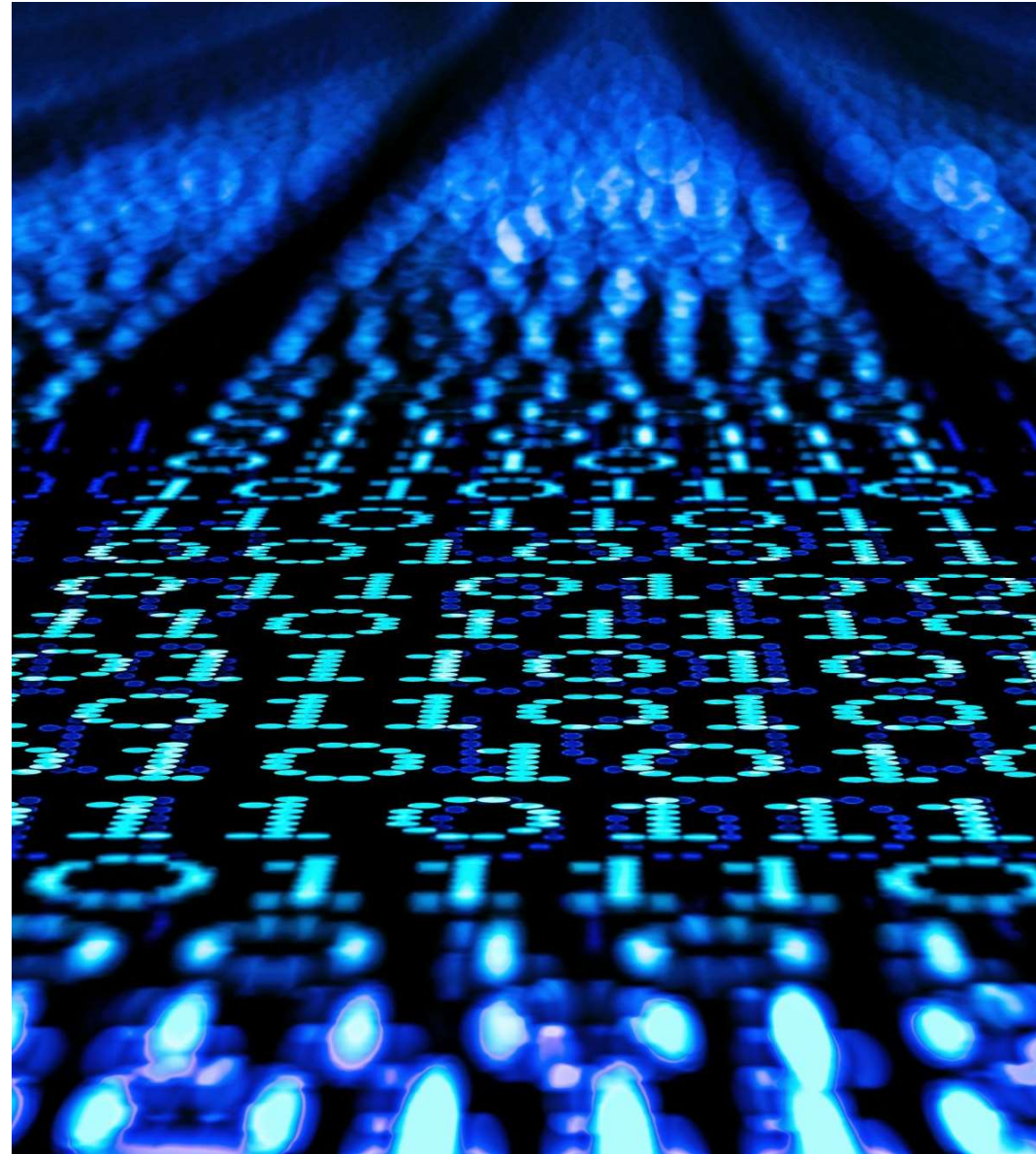

‘try’ / ‘except’

- Use a ‘try’ block to run code which may produce an error
 - ValueError: format entered is not an integer
 - ZeroDivisionError: Number entered results in divide by 0
- The ‘except’ blocks handle these two different kinds of exceptions
 - Any other type of exception will not be caught here

```
x = input("Enter a number: ")
try:
    x = int(x)
    print(10 / x)
except ZeroDivisionError:
    print("You cannot divide by zero!")
except ValueError:
    print(f"'{x}' is not a valid number")
```

‘try’ / ‘except’ code blocks (cont’d)

- Use the ‘try’ block to protect code running within it.
- Use the ‘except’ block(s) to handle errors which occur in the ‘try’ block



'else' clause

- The 'else' clause can be added to to run when no exception is raised
 - Run this with input = 90
 - Run this with input = 0
- Uncaught exceptions will still cause the program to terminate

```
x = input("Enter a number: ")
try:
    x = int(x)
    print(10 / x)
except ValueError:
    print(f"'{x}' is not a valid number")
else:
    print("Something else happened")
```

'finally' clause

- The 'finally' clause can be added execute code regardless of previous exceptions
- It is always executed
 - Try running this code with the following input:
 - 5
 - Hello
 - 0
 - Each case will always print "The end" to the console.

```
x = input("Enter a number: ")
try:
    x = int(x)
    print(10 / x)
except ValueError:
    print(f"'{x}' is not a valid number")
finally:
    print("The end")
```

Raising an exception

- Sometimes you may want to raise a prebuilt exception.
- This is done using the keyword 'raise' followed by the type of exception and any arguments it requires.
- All built-in exceptions are subclasses of the built in 'Exception' class.
- It is possible to define your own exceptions
 - This can be done by creating a sub-class of the standard 'Exception' class
- In the sample to the right, 'bad_function' raises a ValueError exception

```
def good_function():  
    print("I'm good")  
  
def bad_function():  
    raise ValueError("I'm bad")  
  
try:  
    good_function()  
    bad_function()  
except ValueError as e:  
    print(e)  
finally:  
    print("I'm done")
```

Runtime assertion

- Sometimes a programmer will expect a condition to be satisfied
- Use of a runtime 'assert' will raise an `AssertionError` if the condition is not satisfied
 - ``assert`` is a built in keyword.
- Takes two arguments, but not inside parentheses
 - First argument is the condition to test
 - If the condition is False, raises an `AssertionError` with the second argument as the argument to the `AssertionError` constructor
- Code to right will fail assertion if user does not enter an integer value. Try running with values:
 - 76
 - hello

```
x = input("Enter an integer value: ")
try:
    assert x.isdecimal(), f"{x}: not an integer"
    x = int(x)
    print(f"{type(x)}")
except AssertionError as e:
    print(e)
```



Activities

- Write a program that does the following
 - Repeatedly prompt the user for a floating-point number
 - If the value entered is not a floating point number, prompt the user again until the value given can be converted to a float
 - Hint: use the 'float' function inside an exception handler
 - Convert the number given from degrees Fahrenheit to degrees Celsius
 - Print the conversion